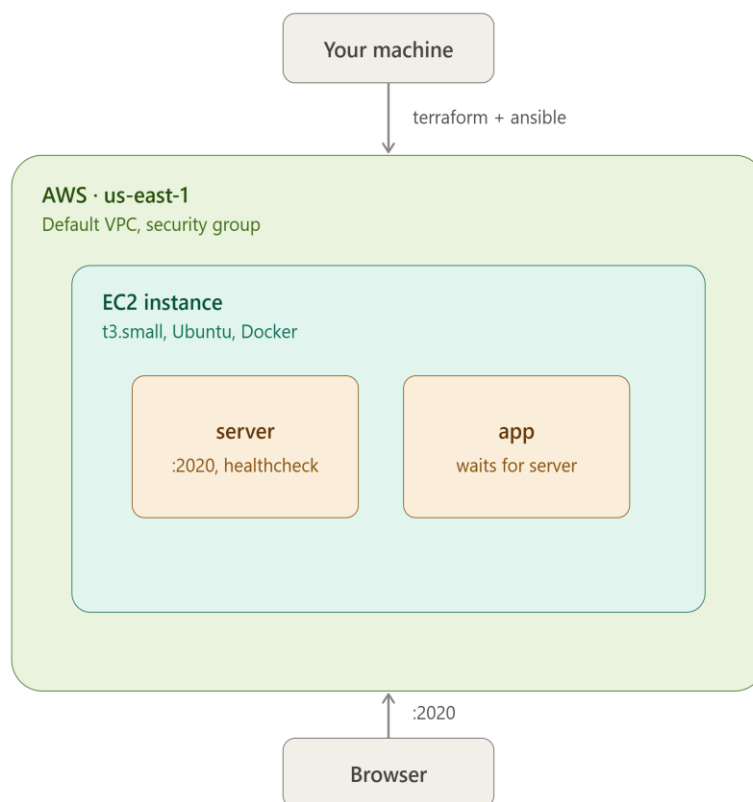


Task 17: Ansible and AWS EC2 Deployment — Documentation

1. Objective

Provision a single EC2 instance with Terraform, then use Ansible to fully configure that instance and deploy Twenty CRM as a set of Docker containers, verify the deployment, and tear the infrastructure back down.

2. Architecture



- Terraform provisions the EC2 instance inside the account's default VPC.
- Ansible then configures that instance and brings up the two containers.
- server is the Twenty CRM all-in-one image; app depends on it being healthy first.

- A browser reaches the app at http://<EC2_PUBLIC_IP>:2020, which routes to server.

3. Tool split

Terraform — what infrastructure exists VPC lookup, security group, EC2 instance.

Ansible — what happens on that infrastructure Packages, Docker, app deployment, verification.

4. Terraform — infrastructure

- `terraform/variables.tf` — region (`us-east-1`), instance type (`t3.small`), one of the two approved AMIs, an existing EC2 key pair name, an SSH-allowed CIDR, the app port (`2020`), and `name_prefix` (used to keep resource names unique in the shared intern AWS account and avoid collisions with other students' resources).
- `terraform/main.tf` — looks up the account's **default VPC** and **default subnet** (task requirement: no custom networking), creates a security group allowing inbound SSH (`22`) and the app port (`2020`), and launches the EC2 instance with a 20GB gp3 root volume (headroom for the Docker image build).
- `terraform/outputs.tf` — prints `ec2_public_ip`, `ec2_public_dns`, `ec2_instance_id`, and a ready-to-open `app_url`.

Commands used:

```
terraform init
terraform apply -var="key_name=twenty-crm-key" -
var="name_prefix=<name>"
```

5. Ansible — configuration and deployment

`ansible/playbook.yml` runs against the host defined in `ansible/inventory.ini` and performs, in order:

1. **Update server packages** — `apt update + dist-upgrade`.
2. **Install Docker and dependencies** — adds Docker's official apt repository (GPG key + repo entry) and installs `docker-ce`, `docker-ce-cli`, `containerd.io`, `docker-`

buildx-plugin, docker-compose-plugin; enables and starts the docker service; adds ubuntu to the docker group.

3. **Swap file** — creates and enables a 2G swapfile (persisted in /etc/fstab). t3.small only has 2GB RAM, and the Docker image build/first boot needs headroom beyond that.
4. **Create application directories** — /opt/twenty-crm, owned by ubuntu.
5. **Deploy Twenty CRM** — clones devops-crm-project (branch fiza-task5) into /opt/twenty-crm.
6. **Configure environment variables** — templates .env from templates/env.j2 (server URL, port, node env) into the app directory.
7. **Build and start via Docker** — builds the images, starts the server container, waits for it to report healthy via `docker inspect`, then starts the remaining containers (see issue #3 below for why this is split into two steps rather than one `up -d --build`).
8. **Restart policy & healthcheck** — both already defined in the repo's `docker-compose.yml` (`restart: unless-stopped`, a `wget`-based healthcheck on /healthz).
9. **Verify the app is running** — waits for port 2020 to accept connections, then polls <http://localhost:2020/healthz> until it returns 200.
10. **Display container status** — `docker compose ps`, printed via Ansible's debug module.
11. **Display application logs** — last 100 lines of `docker compose logs`.

Commands used:

```
ansible twenty_crm -m ping
ansible-playbook playbook.yml
```

6. Verification

- `ansible twenty_crm -m ping` → SUCCESS, pong
- Playbook's own health-endpoint check → 200 on <http://localhost:2020/healthz>
- `docker compose ps` output (captured in the playbook run) showing both containers Up/healthy
- `docker compose logs --tail=100` output captured for reference

- Manual confirmation: http://<EC2_PUBLIC_IP>:2020 opens the Twenty CRM UI in a browser

7. Cleanup

```
cd terraform
terraform destroy -var="key_name=twenty-crm-key" -
var="name_prefix=<name>"
```

8. Deliverables checklist

- [x] EC2 instance created via Terraform (us-east-1, t3.small, approved AMI, default VPC/subnet)
- [x] Ansible inventory populated with the instance's public IP
- [x] Playbook covering: package update, Docker install, app directories, env vars, Docker deployment, restart policy, healthcheck, start, verification, container status, logs
- [x] Playbook run and deployment verified end-to-end
- [x] EC2 instance destroyed via Terraform after completion.